

VexAI1 Code Reference Manual

Generated 2026-05-29 from the current workspace at C:\Users\cloud\Documents\vex-vscode-projects\VexAI1.

This manual explains the autonomous command architecture, major state machines, and function-level responsibilities.

Design preset: compact_reference_guide. The document uses prose sections for concepts, compact function tables for reference, and code snippets for the control paths you are most likely to need to explain.

Table of Contents

- 1. System Overview
- 2. Command Framework
- 3. Hardware, Pose, and Debug Status
- 4. Jetson Vision Data Path
- 5. Field Avoid Zones and FindBlockCommand
- 6. TrackBlockCommand
- 7. Movement Commands
- 8. GPS Filtering and Wall Alignment
- 9. Intake, Color Sorting, Unjamming, and Scoring
- 10. Autonomous Routine Assembly
- 11. Full Function Reference by File
- 12. Explanation Checklist

1. System Overview

This project is a command-based VEX V5 autonomous program. The main loop repeatedly updates odometry, runs a scheduler, applies drivetrain and intake motor powers, and prints diagnostics to the Brain. Higher-level robot behavior is built from Command objects and command groups.

- Hardware definitions live in RobotConfig.cpp / RobotConfig.h.
- Command scheduling lives in Scheduler.h plus the command group classes.
- Autonomous order and tuning constants live mostly in Routines.cpp.
- Vision data comes from the Jetson serial parser, then FindBlockCommand and TrackBlockCommand choose allowed blocks.
- Field movement is handled by drive commands, GPS pose tracking, inertial turning, and wall alignment.
- Intake behavior includes color sorting, block counting, unjamming, and scoring.

Main control loop

```

9: int main() {
10:     jetsonSerial.JetsonSerialSetup();
11:     build_AI_routine();
12:
13:     scheduler.schedule(&AI_ROUTE_ONE);
14:     while(drivetrain_inertial.isCalibrating() || GPSSensor.isCalibrating()){
15:         vex::task::sleep(10);
16:     }
17:     positionTracking.update_raw_pose();
18:     drivebase.set_odom_pose(
19:         positionTracking.get_x(),
20:         positionTracking.get_y(),
21:         positionTracking.get_heading());
22:
23:     while(true){
24:         drivebase.update_odom_pose();
25:         scheduler.run();
26:         drivebase.apply_motor_power();
27:         intake.apply_motor_power();
28:         jetsonSerial.print_block_pos_on_screen();
29:         vex::task::sleep(10);
30:     }
31:
32: }
33:
34:

```

2. Command Framework

Every action implements the Command interface. initialize() sets command state, execute() runs one 10 ms loop slice, isFinished() tells the scheduler when to end it, and end() stops or cleans up.

Base command interface

```

2: class Command{
3:     public:
4:         virtual void initialize() {}
5:         virtual void execute() = 0;
6:         virtual bool isFinished() = 0;
7:         virtual void end() {}
8:         virtual ~Command() {}
9: };

```

Scheduler run loop

```

30:     void run() {
31:         for (int i = 0; i < static_cast<int>(commands.size()); i++) {
32:             Command* command = commands[i];
33:
34:             if (command == nullptr) {
35:                 removeCommand(i);
36:                 i--;
37:                 continue;
38:             }
39:
40:             if (!initialized[i]) {
41:                 command->initialize();
42:                 initialized[i] = true;
43:             }
44:
45:             command->execute();
46:
47:             if (command->isFinished()) {
48:                 command->end();
49:                 removeCommand(i);
50:                 i--;
51:             }
52:         }
53:     }
54:
55:     void cancelAll() {
56:         for (int i = 0; i < static_cast<int>(commands.size()); i++) {
57:             if (commands[i] != nullptr && initialized[i]) {
58:                 commands[i]->end();
59:             }
60:         }
61:     }

```

Function / item	File	How it works
<code>Scheduler::schedule</code>	<code>include/Scheduler.h</code>	Queues a command. Adds a Command pointer and marks it uninitialized so <code>initialize()</code> runs on the next scheduler cycle.
<code>Scheduler::run</code>	<code>include/Scheduler.h</code>	Runs every scheduled command. Initializes unstarted commands, calls <code>execute()</code> , then calls <code>end()</code> and removes commands whose <code>isFinished()</code> returns true.
<code>SequentialCommandGroup::execute</code>	<code>src/SequentialCommandGroup.cpp</code>	Runs one command at a time. Initializes the current command, executes it until finished, ends it, then advances to the next command.
<code>ParallelCommandGroup::execute</code>	<code>src/ParallelCommandGroup.cpp</code>	Runs commands together. Executes every child each loop and finishes only when all children are finished.
<code>ParallelDeadlineGroup::execute</code>	<code>src/ParallelDeadlineGroup.cpp</code>	Runs helpers until a deadline ends. The deadline command controls group completion; helper commands run in parallel and are ended when the group ends.
<code>RaceCommandGroup::execute</code>	<code>src/RaceCommandGroup.cpp</code>	First command to finish wins. Executes every child and finishes the group as soon as any child reports done.
<code>ConditionalCommandGroup::initialize</code>	<code>src/ConditionalCommandGroup.cpp</code>	Chooses one branch. Evaluates a boolean function and initializes either the true command or false command.
<code>RepeatForeverCommandGroup::execute</code>	<code>src/RepeatForeverCommandGroup.cpp</code>	Loops a command group forever. Runs the wrapped command; when it finishes, calls <code>end()</code> and <code>initialize()</code> to restart it.

3. Hardware, Pose, and Debug Status

`RobotConfig.cpp` owns the VEX hardware objects. `Drivetrain` stores the requested motor powers and applies them later in the main loop. `PositionTracking` reads GPS, applies the physical GPS sensor offset, and exposes robot-center field coordinates to commands.

GPS sensor offset correction

```

28: float PositionTracking::get_raw_x(){
29:     float headingRadians = get_raw_heading() * PI / 180.0f;
30:
31:     float sensorOffsetX =
32:         GPS_OFFSET_RIGHT_INCHES * std::cos(headingRadians) +
33:         GPS_OFFSET_FORWARD_INCHES * std::sin(headingRadians);
34:
35:     return GPSSensor.xPosition(inches) - sensorOffsetX;
36: }
37: float PositionTracking::get_raw_y(){
38:     float headingRadians = get_raw_heading() * PI / 180.0f;
39:
40:     float sensorOffsetY =
41:         -GPS_OFFSET_RIGHT_INCHES * std::sin(headingRadians) +
42:         GPS_OFFSET_FORWARD_INCHES * std::cos(headingRadians);
43:
44:     return GPSSensor.yPosition(inches) - sensorOffsetY;
45: }
46:
47: float PositionTracking::get_raw_heading(){
48:     return normalize_heading(GPSSensor.heading(degrees) + 180);
49: }
50:
51: void PositionTracking::update_raw_pose(){
52:     gpsPose.x = get_raw_x();
53:     gpsPose.y = get_raw_y();
54:     gpsPose.heading = get_raw_heading();

```

Function / item	File	How it works
Drivetrain::set_odom_pose	src/Drivetrain.cpp	Sets x/y/theta pose. Stores pose and calls set_heading_degrees() so the inertial heading is aligned to the pose heading.
Drivetrain::set_heading_degrees	src/Drivetrain.cpp	Safely writes inertial heading. Rejects non-finite headings, normalizes to 0..359.999, then calls drivetrain_inertial.setHeading().
Drivetrain::set_drive_power	src/Drivetrain.cpp	Stores requested power. Does not spin motors immediately; main() calls apply_motor_power() once per loop.
Drivetrain::apply_motor_power	src/Drivetrain.cpp	Applies drivetrain output. Spins the left and right motor groups using the latest stored percentages.
PositionTracking::get_raw_x/y	src/PositionTracking.cpp	Returns robot-center GPS pose. Reads the GPS sensor position and subtracts the rotated offset: 4 inches right, 7 inches behind the robot center.
PositionTracking::update_raw_pose	src/PositionTracking.cpp	Refreshes cached pose. Stores corrected x/y plus normalized GPS heading in gpsPose.
CommandStatus setters	src/CommandStatus.cpp	Stores debug display values. Holds current command label and last wall alignment distance for Brain screen output.
JetsonSerial::print_block_pos_on_screen	src/JetsonSerial.cpp	Brain diagnostic display. Prints selected block, red count, vision errors, wall distance, GPS, IMU, and current command status.

Brain display rows

```

238: // void JetsonSerial::print_block_pos_on_screen(){
239: //     Brain.Screen.setCursor(1, 1);
240: //     Brain.Screen.print("Block X: %d      ", block_x_pos);
241:
242: //     Brain.Screen.setCursor(2, 1);
243: //     Brain.Screen.print("Block Y: %d      ", block_y_pos);
244:
245: //     Brain.Screen.setCursor(3, 1);
246: //     Brain.Screen.print("X Error: %d      ", trackBlockRawVar.xError);
247:
248: //     Brain.Screen.setCursor(4, 1);
249: //     Brain.Screen.print("Y Error: %d      ", trackBlockRawVar.yError);
250:
251: //     Brain.Screen.setCursor(5, 1);
252: //     Brain.Screen.print("Visible: %d      ", trackBlockRawVar.targetVisible);
253:
254: //     Brain.Screen.setCursor(6, 1);
255: //     Brain.Screen.print("Locked: %d      ", trackBlockRawVar.trackingLocked);
256: // }
257:
258: #include "JetsonSerial.h"
259: #include <stdio.h>
260: #include "RobotConfig.h"
261: #include "CommandStatus.h"
262: #include <cstring>
263: #include <cstdlib>
264: #include <algorithm>
265:
266: void JetsonSerial::JetsonSerialSetup(){
267:     serialPTR = fopen("/dev/serial1", "r");
268:
269:     if(!serialPTR){
270:         Brain.Screen.print("Failed to open dev/serial1");
271:         return;
272:     }

```

4. Jetson Vision Data Path

The Jetson sends either 0 or a semicolon-separated list of block coordinates. JetsonSerial parses that into arrays, then raw find/track functions select a block. The higher-level find and track commands now filter those raw candidates through field avoid zones before accepting a block.

Multi-block serial parser

```

18: // void JetsonSerial::update_block_pose(){
19: //     if(fgets(buffer, sizeof(buffer), serialPTR) != NULL){
20: //         len = std::strlen(buffer);
21: //         if(len > 0 && (buffer[len-1] == '\r' || buffer[len-1] == '\n')){
22: //             buffer[len-1] = '\0';
23: //         }
24: //         cmd_str = buffer;
25: //     }
26: //     std::strcpy(pos_string, cmd_str.c_str());
27: //     for(int i = 0; i < 10; i++){
28: //         if(pos_string[i] == ','){
29: //             comma_index = i;
30: //             continue;
31: //         } else if (pos_string[i] == '\0'){
32: //             null_index = i;
33: //             break;
34: //         }
35: //     }
36: //     tempX = cmd_str.substr(0, comma_index);
37: //     tempY = cmd_str.substr(comma_index + 1, null_index - comma_index - 1);
38:
39: //     block_x_pos = std::atoi(tempX.c_str());
40: //     block_y_pos = std::atoi(tempY.c_str());
41: // }
42:
43:
44: // void JetsonSerial::find_block_raw_init(FindBlockRawConfig config){
45: //     findBlockRawConfig = config;
46: //     FindBlockRawVar& var = findBlockRawVar;
47: //     var.lastBlockXPos = -1;
48: //     var.lastBlockYPos = -1;
49: //     var.lastBlockXDistance = 0;
50: //     var.lastBlockYDistance = 0;
51: //     var.sequentialBlocksCount = 0;
52: //     var.x_is_stable = false;
53: //     var.y_is_stable = false;
54: // };
55:
56: // bool JetsonSerial::find_block_raw_step(){
57: //     FindBlockRawVar& var = findBlockRawVar;
58: //     FindBlockRawConfig& conf = findBlockRawConfig;
59:
60: //     update_block_pose();
61:
62: //     if(block_x_pos < 0 || block_y_pos < 0){
63: //         var.sequentialBlocksCount = 0;
64: //         var.lastBlockXPos = -1;
65: //         var.lastBlockYPos = -1;
66:
67: //         var.lastBlockXDistance = 0;
68: //         var.lastBlockYDistance = 0;
69:
70: //         var.x_is_stable = false;
71: //         var.y_is_stable = false;
72:
73: //         return false;
74: //     }
75:
76: //     if(var.lastBlockXPos == -1 || var.lastBlockYPos == -1){
77: //         var.lastBlockXPos = block_x_pos;
78: //         var.lastBlockYPos = block_y_pos;
79:
80: //         var.lastBlockXDistance = 0;
81: //         var.lastBlockYDistance = 0;
82:
83: //         var.sequentialBlocksCount = 0;
84:
85: //         var.x_is_stable = false;
86: //         var.y_is_stable = false;
87:

```

Generic closest-block helper

```

324: bool JetsonSerial::select_block_closest_to(int targetX, int targetY){
325:     if(block_count <= 0){
326:         block_x_pos = -1;
327:         block_y_pos = -1;
328:         return false;
329:     }
330:
331:     int bestIndex = 0;
332:     long bestDistance = 2147483647;
333:
334:     for(int i = 0; i < block_count; i++){
335:         long dx = block_x_positions[i] - targetX;
336:         long dy = block_y_positions[i] - targetY;
337:
338:         // Manhattan distance is good enough here and avoids using sqrt.
339:         long distance = std::abs(dx) + std::abs(dy);
340:
341:         if(distance < bestDistance){
342:             bestDistance = distance;
343:             bestIndex = i;
344:         }
345:     }
346:
347:     return select_block_index(bestIndex);
348: }
349:

```

Function / item	File	How it works
clear_blocks	src/JetsonSerial.cpp	Resets vision frame state. Clears selected position and every stored block coordinate before parsing a new serial message.
parse_xy_pair	src/JetsonSerial.cpp	Parses one coordinate pair. Splits a token at a comma and converts x/y text to integers.
select_block_index	src/JetsonSerial.cpp	Selects an array item. Copies block_x_positions[index] and block_y_positions[index] into the active selected block.
select_block_closest_to	src/JetsonSerial.cpp	Generic unfiltered chooser. Finds the block closest to a target pixel by Manhattan distance. This does not know about avoid zones.
update_block_pose	src/JetsonSerial.cpp	Reads one serial frame. Supports old x,y format and new count;x,y;x,y format; after parsing it selects index 0 by default.
find_block_raw_step	src/JetsonSerial.cpp	Raw stable-block detector. Checks whether selected block movement is small for enough sequential frames. The higher-level FindBlockCommand replaces selection with avoid-zone-aware selection.
track_block_raw_step	src/JetsonSerial.cpp	Raw tracker. Maintains a target lock using jump thresholds and lost-frame tolerance. The higher-level commands now use allowed-only tracking helpers instead of this generic method when avoid zones matter.
getXError / getYError	src/JetsonSerial.cpp	Vision error accessors. Return the x and y errors stored in trackBlockRawVar for PID control.

5. Field Avoid Zones and FindBlockCommand

FindBlockCommand is responsible for scanning for a legal block, centering on it, and rejecting blocks whose estimated field position falls inside an avoid zone. Normal search is now rotation in place to the right. The circular arc unstuck sequence only starts if the robot is supposed to rotate but heading does not change.

Allowed block selection

```
579:     bool selectAllowedBlockClosestTo(int targetX, int targetY) {
580:         int bestIndex = -1;
581:         long bestDistanceSquared = 0;
582:
583:         for (int i = 0; i < jetson.block_count; i++) {
584:             int candidateX = jetson.block_x_positions[i];
585:             int candidateY = jetson.block_y_positions[i];
586:
587:             if (candidateX < 0 || candidateY < 0) {
588:                 continue;
589:             }
590:
591:             if (candidateBlockIsInsideAvoidZone(candidateX, candidateY)) {
592:                 continue;
593:             }
594:
595:             long dx = candidateX - targetX;
596:             long dy = candidateY - targetY;
597:             long distanceSquared = dx * dx + dy * dy;
598:
599:             if (bestIndex < 0 || distanceSquared < bestDistanceSquared) {
600:                 bestIndex = i;
601:                 bestDistanceSquared = distanceSquared;
602:             }
603:         }
604:
605:         if (bestIndex < 0) {
606:             jetson.block_x_pos = -1;
607:             jetson.block_y_pos = -1;
608:             return false;
609:         }
610:
611:         jetson.block_x_pos = jetson.block_x_positions[bestIndex];
612:         jetson.block_y_pos = jetson.block_y_positions[bestIndex];
613:         return true;
614:     }
615:
616:     bool findAllowedBlockRawStep() {
617:         FindBlockRawVar& var = jetson.findBlockRawVar;
```

Allowed-only tracking while centering

```

706:     bool trackAllowedBlockRawStep() {
707:         TrackBlockRawVar& var = jetson.trackBlockRawVar;
708:         TrackBlockRawConfig& conf = trackingConfig;
709:
710:         jetson.update_block_pose();
711:
712:         if (jetson.block_count <= 0) {
713:             return updateAllowedTrackingLostFrame();
714:         }
715:
716:         int targetX =
717:             var.lastBlockXPos >= 0 ?
718:             var.lastBlockXPos :
719:             conf.cameraCenterX;
720:
721:         int targetY =
722:             var.lastBlockYPos >= 0 ?
723:             var.lastBlockYPos :
724:             conf.cameraCenterY;
725:
726:         if (!selectAllowedBlockClosestTo(targetX, targetY)) {
727:             return updateAllowedTrackingLostFrame();
728:         }
729:
730:         var.targetVisible = true;
731:
732:         if (!var.trackingLocked) {
733:             var.lastBlockXPos = jetson.block_x_pos;
734:             var.lastBlockYPos = jetson.block_y_pos;
735:
736:             var.xJump = 0;
737:             var.yJump = 0;
738:
739:             var.xJumpValid = true;
740:             var.yJumpValid = true;
741:
742:             var.xError = jetson.block_x_pos - conf.cameraCenterX;
743:             var.yError = conf.cameraCenterY - jetson.block_y_pos;
744:
745:             var.lostFrameCount = 0;
746:             var.trackingLocked = true;
747:             var.targetVisible = true;
748:
749:             return true;
750:         }
751:
752:         var.xJump = jetson.block_x_pos - var.lastBlockXPos;
753:         var.yJump = jetson.block_y_pos - var.lastBlockYPos;
754:
755:         var.xJumpValid =
756:             std::abs(var.xJump) <= conf.maxTrackingXJump;
757:
758:         var.yJumpValid =
759:             std::abs(var.yJump) <= conf.maxTrackingYJump;
760:
761:         if (!var.xJumpValid || !var.yJumpValid) {
762:             return updateAllowedTrackingLostFrame();
763:         }
764:
765:         var.lastBlockXPos = jetson.block_x_pos;
766:         var.lastBlockYPos = jetson.block_y_pos;
767:
768:         var.xError = jetson.block_x_pos - conf.cameraCenterX;
769:         var.yError = conf.cameraCenterY - jetson.block_y_pos;
770:
771:         var.lostFrameCount = 0;
772:         var.trackingLocked = true;
773:         var.targetVisible = true;
774:
775:         return true;
776:     }
777:
778:     void resetCenteringStuckMonitor() {
779:         lastCenteringCheckTime = master_timer.time(msec);

```

Search movement and stuck detection

```

120: // void applySearchMovement() {
121: //     if (currentSearchMovement == SEARCH_TURN_RIGHT) {
122: //         drivetrain.set_drive_power(config.searchSpeed, -config.searchSpeed);
123: //     }
124: //     else if (currentSearchMovement == SEARCH_TURN_LEFT) {
125: //         drivetrain.set_drive_power(-config.searchSpeed, config.searchSpeed);
126: //     }
127: //     else if (currentSearchMovement == SEARCH_REVERSE) {
128: //         drivetrain.set_drive_power(-config.reverseSpeed, -config.reverseSpeed);
129: //     }
130: //     else if (currentSearchMovement == SEARCH_FORWARD) {
131: //         drivetrain.set_drive_power(config.forwardSpeed, config.forwardSpeed);
132: //     }
133: // }
134:
135: // void updateStuckDetection() {
136: //     float now = master_timer.time(msec);
137:
138: //     float movementElapsed = now - movementStartTime;
139:
140: //     // Reverse should only be used as an unsticking move,
141: //     // not forever.
142: //     if (currentSearchMovement == SEARCH_REVERSE &&
143: //         movementElapsed >= config.maxReverseTime) {
144:
145: //         goToNextSearchMovement();
146: //         return;
147: //     }
148:
149: //     // Forward should also only be used briefly.
150: //     if (currentSearchMovement == SEARCH_FORWARD &&
151: //         movementElapsed >= config.maxForwardTime) {
152:
153: //         goToNextSearchMovement();
154: //         return;
155: //     }
156:
157: //     // Only check stuck status every stuckCheckTime.
158: //     if (now - lastStuckCheckTime < config.stuckCheckTime) {
159: //         return;
160: //     }
161:
162: //     float currentHeading = drivetrain.get_heading_degrees();
163: //     float currentEncoderPosition = drivetrain.get_left_front_motor_position();
164:
165: //     float headingChange = getHeadingChange(currentHeading, lastHeading);
166: //     float encoderChange = currentEncoderPosition - lastEncoderPosition;
167:
168: //     bool madeProgress = false;
169:
170: //     if (currentSearchMovement == SEARCH_TURN_RIGHT ||
171: //         currentSearchMovement == SEARCH_TURN_LEFT) {
172:
173: //         madeProgress =
174: //             std::fabs(headingChange) >= config.stuckHeadingChangeThreshold;
175: //     }
176: //     else if (currentSearchMovement == SEARCH_REVERSE ||
177: //         currentSearchMovement == SEARCH_FORWARD) {
178:
179: //         madeProgress =
180: //             std::fabs(encoderChange) >= config.stuckEncoderChangeThreshold;
181: //     }
182:
183: //     if (!madeProgress) {
184: //         goToNextSearchMovement();
185: //         return;
186: //     }
187:
188: //     // Robot moved enough, so keep doing the same movement.
189: //     lastStuckCheckTime = now;
190: //     lastHeading = currentHeading;
191: //     lastEncoderPosition = currentEncoderPosition;
192: // }
193:
194: // public:
195: //     FindBlockCommand(
196: //         Drivetrain& drivetrain,
197: //         JetsonSerial& jetson,

```

FindBlockCommand execute state machine

```

245: // void execute() override {
246: //     if (currentState == SEARCHING_FOR_BLOCK) {
247: //         foundBlock = jetson.find_block_raw_step();
248: //
249: //         if (!foundBlock) {
250: //             applySearchMovement();
251: //             updateStuckDetection();
252: //             return;
253: //         }
254: //
255: //         drivetrain.set_drive_power(0, 0);
256: //
257: //         jetson.track_block_raw_init(trackingConfig);
258: //
259: //         currentState = CENTERING_BLOCK;
260: //         return;
261: //     }
262: //
263: //     else if (currentState == CENTERING_BLOCK) {
264: //         trackingValid = jetson.track_block_raw_step();
265: //
266: //         if (!trackingValid) {
267: //             jetson.find_block_raw_init(findingConfig);
268: //
269: //             currentState = SEARCHING_FOR_BLOCK;
270: //             currentSearchMovement = SEARCH_TURN_RIGHT;
271: //
272: //             drivetrain.set_drive_power(0, 0);
273: //
274: //             resetStuckMonitor();
275: //
276: //             return;
277: //         }
278: //
279: //         xError = jetson.getXError();
280: //         xDerivativeError = xError - xPrevError;
281: //
282: //         angularSpeed =
283: //             xError * PID.angular_kP +
284: //             xDerivativeError * PID.angular_kD;
285: //
286: //         if (std::fabs(angularSpeed) > config.maxSpeed) {
287: //             if (angularSpeed > 0) {
288: //                 angularSpeed = config.maxSpeed;
289: //             } else {
290: //                 angularSpeed = -config.maxSpeed;
291: //             }
292: //         }
293: //
294: //         drivetrain.set_drive_power(angularSpeed, -angularSpeed);
295: //
296: //         xPrevError = xError;
297: //
298: //         if (std::abs(xError) <= config.centeringAcceptableX) {
299: //             drivetrain.set_drive_power(0, 0);
300: //             currentState = FIND_BLOCK_DONE;
301: //             finished = true;
302: //         }
303: //
304: //         return;
305: //     }
306: //
307: //     else if (currentState == FIND_BLOCK_DONE) {
308: //         drivetrain.set_drive_power(0, 0);
309: //         finished = true;
310: //         return;
311: //     }
312: // }
313: //
314: // bool isFinished() override {
315: //     return finished;
316: // }
317: //
318: // void end() override {
319: //     drivetrain.set_drive_power(0, 0);
320: // }
321: // };
322: #pragma once
323:
324: #include "Command.h"

```

```

325: #include "JetsonSerial.h"
326: #include "RobotConfig.h"
327: #include "Drivetrain.h"
328: #include "PositionTracking.h"
329: #include "FieldAvoidZone.h"
330: #include "CommandStatus.h"
331:
332: #include <cmath>
333:
334: struct findingBlocksConfig {
335:     float searchSpeed;
336:     float maxSpeed;
337:     int centeringAcceptableX;
338:
339:     // Anti-stuck movement speeds
340:     float reverseSpeed;
341:     float forwardSpeed;
342:     float turnSpeed;
343:
344:     // How often to check if the robot is stuck, in milliseconds
345:     float stuckCheckTime;
346:
347:     // Minimum heading change needed while turning
348:     float stuckHeadingChangeThreshold;
349:
350:     // Minimum encoder change needed while driving forward/reverse
351:     float stuckEncoderChangeThreshold;
352:
353:     // Maximum time to run backward/forward search arcs, in milliseconds
354:     float maxReverseTime;
355:     float maxForwardTime;
356:
357:     // Used to estimate a block's field position before it is perfectly centered.
358:     float cameraHorizontalFovDegrees;
359:
360:     // Field rectangles to avoid
361:     FieldAvoidZone avoidZones[9];
362:
363:     // How long to turn away from an avoided block, in milliseconds
364:     float avoidTurnTime;
365:
366:     // How many bad tracking frames to tolerate while centering
367:     int maxCenteringDroppedFrames;
368:
369:     // Minimum x-error improvement needed while centering on a block
370:     int centeringXProgressThreshold;
371: };
372:
373: enum findingBlockState {
374:     SEARCHING_FOR_BLOCK,
375:     CENTERING_BLOCK,
376:     FIND_BLOCK_DONE
377: };
378:
379: enum searchMovementState {
380:     SEARCH_FORWARD_RIGHT,
381:     SEARCH_FORWARD_LEFT,
382:     SEARCH_BACK_RIGHT,
383:     SEARCH_BACK_LEFT
384: };

```

Function / item	File	How it works
estimateBlockDistanceFromPixelY	include/FindBlockCommand.h	Approximates distance from pixel y. Uses a polynomial mapping from image y to estimated inches from the robot.
estimateBlockFieldPosition	include/FindBlockCommand.h	Projects a block onto the field. Uses robot GPS x/y/heading, camera center error, and horizontal FOV to estimate block field coordinates.
pointInsideAvoidZone	include/FindBlockCommand.h	Rectangle test. Normalizes min/max x/y and returns true if a field point is inside an enabled zone.
blockIsInsideAnyAvoidZone	include/FindBlockCommand.h	Checks all 9 zones. Returns true if the estimated block field position is in any configured avoid rectangle.
candidateBlockIsInsideAvoidZone	include/FindBlockCommand.h	Pixel candidate filter. Estimates the candidate field location and calls blockIsInsideAnyAvoidZone().

Function / item	File	How it works
selectAllowedBlockClosestTo	include/FindBlockCommand.h	Legal block chooser. Scans every visible block, skips rejected zones, and chooses the allowed block closest to the target pixel. If all blocks are rejected, it clears the selected block.
findAllowedBlockRawStep	include/FindBlockCommand.h	Stable legal-block find step. Updates one Jetson frame, selects only legal blocks, checks stability across sequential frames, and returns true once the block is stable enough.
trackAllowedBlockRawStep	include/FindBlockCommand.h	Legal-block tracker. During centering, it keeps choosing from allowed blocks only, so a rejected block cannot steal the target lock.
applySearchMovement	include/FindBlockCommand.h	Search or unstuck motion. If not unstucking, rotates in place right. If unstucking, runs forward/right, forward/left, back/right, back/left arcs.
updateStuckDetection	include/FindBlockCommand.h	Starts search unstuck only when needed. While normally searching, it checks heading change only. If heading is not changing, it starts the arc sequence.
centeringMadeProgress	include/FindBlockCommand.h	Detects centering stall. While rotating to center a block, expects either heading movement or x-error improvement.
execute	include/FindBlockCommand.h	Main state machine. SEARCHING_FOR_BLOCK rotates and finds a stable allowed block. CENTERING_BLOCK tracks allowed blocks, centers by x error, and finishes when centered.

6. TrackBlockCommand

TrackBlockCommand drives toward a selected block using x error for turning and y error for forward motion. Its allowed-only tracking step mirrors the find command: it filters rejected field zones first, then picks the closest acceptable block to the last tracked position.

Track allowed-only selector

```
402:     bool selectAllowedBlockClosestTo(int targetX, int targetY) {
403:         int bestIndex = -1;
404:         long bestDistanceSquared = 0;
405:
406:         for (int i = 0; i < jetson.block_count; i++) {
407:             int candidateX = jetson.block_x_positions[i];
408:             int candidateY = jetson.block_y_positions[i];
409:
410:             if (candidateX < 0 || candidateY < 0) {
411:                 continue;
412:             }
413:
414:             if (candidateBlockIsInsideAvoidZone(candidateX, candidateY)) {
415:                 continue;
416:             }
417:
418:             long dx = candidateX - targetX;
419:             long dy = candidateY - targetY;
420:             long distanceSquared = dx * dx + dy * dy;
421:
422:             if (bestIndex < 0 || distanceSquared < bestDistanceSquared) {
423:                 bestIndex = i;
424:                 bestDistanceSquared = distanceSquared;
425:             }
426:         }
427:
428:         if (bestIndex < 0) {
429:             jetson.block_x_pos = -1;
430:             jetson.block_y_pos = -1;
431:             return false;
432:         }
433:
434:         jetson.block_x_pos = jetson.block_x_positions[bestIndex];
435:         jetson.block_y_pos = jetson.block_y_positions[bestIndex];
436:         return true;
437:     }
438:
439:     bool updateAllowedTrackingLostFrame() {
440:         TrackBlockRawVar& var = jetson.trackBlockRawVar;
```

TrackBlockCommand execute loop

```

79: //      void execute() override{
80: //          trackingValid = jetson.track_block_raw_step();
81:
82: //          if(!trackingValid){
83: //              drivetrain.set_drive_power(0,0);
84: //              finished = true;
85: //              return;
86: //          }
87:
88: //          xError = jetson.getXError();
89: //          xDerivativeError = xError - xPrevError;
90: //          if(abs(xError) <= PID.angular_integral_windup_threshold){
91: //              xIntegralError += xError;
92: //          } else {
93: //              xIntegralError = 0;
94: //          }
95:
96:
97: //          angularSpeed = PID.angular_kP * xError + PID.angular_kI * xIntegralError +
PID.angular_kD * xDerivativeError;
98: //          if(fabs(angulaSpeed) > config.maxAngularSpeed){
99: //              if(angulaSpeed > 0){
100: //                  angularSpeed = config.maxAngularSpeed;
101: //              } else {
102: //                  angularSpeed = -config.maxAngularSpeed;
103: //              }
104: //          }
105:
106:
107: //          yError = jetson.getYError();
108: //          yDerivativeError = yError - yPrevError;
109: //          if(abs(yError) <= PID.linear_integral_windup_threshold){
110: //              yIntegralError += yError;
111: //          } else {
112: //              yIntegralError = 0;
113: //          }
114:
115: //          linearSpeed = PID.linear_kP * yError + PID.linear_kI * yIntegralError +
PID.linear_kD * yDerivativeError;
116:
117: //          if(fabs(linearSpeed) > config.maxLinearSpeed){
118: //              if(linearSpeed > 0){
119: //                  linearSpeed = config.maxLinearSpeed;
120: //              } else {
121: //                  linearSpeed = -config.maxLinearSpeed;
122: //              }
123: //          }
124:
125: //          xPrevError = xError;
126: //          yPrevError = yError;
127:
128: //          drivetrain.set_drive_power(linearSpeed + angularSpeed, linearSpeed -
angularSpeed);
129:
130: //          xCentered = abs(xError) <= config.acceptableXError;
131: //          yCloseEnough = abs(yError) <= config.acceptableYError;
132:
133: //          if(xCentered && yCloseEnough){
134: //              drivetrain.set_drive_power(0,0);
135: //              finished = true;
136: //          }
137:
138: //      }
139:
140: //      bool isFinished() override{
141: //          return finished;
142: //      }
143:
144: //      void end() override{
145: //          drivetrain.set_drive_power(0,0);
146: //      }
147: // };
148:
149: #pragma once
150:
151: #include "Command.h"
152: #include "JetsonSerial.h"
153: #include "Drivetrain.h"
154: #include "RobotConfig.h"
155: #include "PositionTracking.h"

```

```

156: #include "FieldAvoidZone.h"
157: #include "CommandStatus.h"
158:
159: #include <cmath>
160:
161: struct TrackingBlocksConfig {
162:     int acceptableYError;
163:     int acceptableXError;
164:
165:     float maxLinearSpeed;
166:     float maxAngularSpeed;
167:
168:     // Anti-stuck settings
169:     float stuckCheckTime; // ms
170:     float stuckEncoderChangeThreshold; // motor degrees
171:     float stuckHeadingChangeThreshold; // degrees
172:
173:     float reverseSpeed;
174:     float forwardSpeed;
175:     float turnSpeed;
176:
177:     float maxReverseTime; // ms
178:     float maxForwardTime; // ms
179:
180:     float minLinearSpeedForStuckCheck;
181:     float minYErrorProgress;
182:
183:     // How many raw tracking failures to tolerate before giving up.
184:     int maxTrackingDroppedFrames;
185:
186:     // How many full unstuck escape cycles to try before giving up.
187:     int maxUnstuckAttempts;
188:
189:     // Used to estimate a block's field position before it is perfectly centered.
190:     float cameraHorizontalFovDegrees;
191:
192:     // Avoid-zone settings
193:     FieldAvoidZone avoidZones[9];
194: };
195:
196: enum TrackBlockState {
197:     TRACKING_NORMAL,
198:     TRACKING_UNSTUCK_FORWARD_RIGHT,

```

Function / item	File	How it works
failStuck	include/TrackBlockCommand.h	Ends with stuck failure. Stops drive power, sets result to TRACK_BLOCK_STUCK_FAILED, and finishes.
estimateBlockFieldPosition	include/TrackBlockCommand.h	Same projection as find. Uses GPS pose plus camera bearing to estimate where the block is on the field.
trackAllowedBlockRawStep	include/TrackBlockCommand.h	Avoid-zone-aware tracking. Updates Jetson frame, picks only accepted blocks, applies jump thresholds, and maintains lost-frame behavior.
robotMadeProgressWhileTracking	include/TrackBlockCommand.h	Progress detector. Accepts progress from encoder movement, heading movement, or enough y-error change.
runUnstuckState	include/TrackBlockCommand.h	Circular arc escape. Runs forward/right, forward/left, back/right, back/left, then returns to normal tracking.
execute	include/TrackBlockCommand.h	PID block approach. Tracks allowed block, rejects avoid-zone targets, calculates angular and linear PID, starts unstuck if y progress stalls, and succeeds when x/y are within thresholds.
wasSuccessful / wasAvoided / lostTracking	include/TrackBlockCommand.h	Result helpers. Expose the result to Routines.cpp so the robot can decide whether to grab or skip.

7. Movement Commands

Movement commands share the same pattern: initialize local error/PID state, repeatedly read sensors, compute PID output, clamp speeds, optionally run unstuck logic, and finish on settle time or timeout.

Drive-to-point target heading and distance

```
175:     void updatePositionAndTarget() {
176:         positionTracking.update_raw_pose();
177:
178:         currentX = positionTracking.get_x();
179:         currentY = positionTracking.get_y();
180:
181:         float xError = target.target_x - currentX;
182:         float yError = target.target_y - currentY;
183:
184:         const float PI = 3.14159265f;
185:         targetHeading =
186:             normalizeHeading(std::atan2(xError, yError) * 180.0f / PI);
187:
188:         if (config.drive_direction == DRIVE_TO_POINT_DRIVE_BACKWARD) {
189:             targetHeading = normalizeHeading(targetHeading + 180.0f);
190:         }
191:
192:         distanceError = std::sqrt(xError * xError + yError * yError);
193:         headingErrorValue =
194:             getHeadingError(targetHeading, drivetrain.get_heading_degrees());
195:     }
196:
197:     void resetStuckMonitor() {
```

Drive-to-point execute state machine

```

524: void execute() override {
525:     float now = master_timer.time(msec);
526:
527:     updatePositionAndTarget();
528:
529:     if (lastStuckCheckTime == 0) {
530:         resetStuckMonitor();
531:     }
532:
533:     if (now >= endTime) {
534:         timedOut = true;
535:         finished = true;
536:         drivetrain.set_drive_power(0, 0);
537:         return;
538:     }
539:
540:     if (currentState == DRIVE_TO_POINT_UNSTUCK_FORWARD_RIGHT ||
541:         currentState == DRIVE_TO_POINT_UNSTUCK_FORWARD_LEFT ||
542:         currentState == DRIVE_TO_POINT_UNSTUCK_BACK_RIGHT ||
543:         currentState == DRIVE_TO_POINT_UNSTUCK_BACK_LEFT) {
544:
545:         runUnstuckState();
546:         return;
547:     }
548:
549:     float angularSpeed = getAngularSpeed();
550:
551:     if (currentState == DRIVE_TO_POINT_POINTING) {
552:         bool pointedAtTarget =
553:             updateSettledTimer(
554:                 std::fabs(headingErrorValue) <= config.heading_acceptable_error,
555:                 pointingTimerSet,
556:                 pointingSettleEndTime,
557:                 config.pointing_settle_time,
558:                 now
559:             );
560:
561:         if (pointedAtTarget) {
562:             currentState = DRIVE_TO_POINT_DRIVING;
563:             pointingTimerSet = false;
564:             positionTimerSet = false;
565:             linearPreviousError = distanceError;
566:             linearIntegralError = 0;
567:             resetStuckMonitor();
568:             endTime = now + config.max_time;
569:         } else {
570:             if (!robotMadeProgressWhilePointing(angularSpeed)) {
571:                 startUnstuckForwardRight();
572:                 return;
573:             }
574:
575:             drivetrain.set_drive_power(angularSpeed, -angularSpeed);
576:             return;
577:         }
578:     }
579:
580:     if (currentState == DRIVE_TO_POINT_DRIVING) {
581:         bool atPosition =
582:             updateSettledTimer(
583:                 distanceError <= config.position_acceptable_error,
584:                 positionTimerSet,
585:                 positionSettleEndTime,
586:                 config.position_settle_time,
587:                 now
588:             );
589:
590:         if (atPosition) {
591:             currentState = DRIVE_TO_POINT_DONE;
592:             finished = true;
593:             drivetrain.set_drive_power(0, 0);
594:             return;
595:         }
596:
597:         float linearSpeed = getLinearSpeed();
598:
599:         if (!robotMadeProgressWhileDriving(linearSpeed)) {
600:             startUnstuckForwardRight();
601:             return;
602:         }
603:

```

```

604:         float leftPower = linearSpeed + angularSpeed;
605:         float rightPower = linearSpeed - angularSpeed;
606:         scaleDrivePower(leftPower, rightPower);
607:
608:         drivetrain.set_drive_power(leftPower, rightPower);
609:         return;
610:     }
611:
612:     drivetrain.set_drive_power(0, 0);
613:     finished = true;
614: }
615:
616: bool isFinished() override {
617:     return finished;
618: }

```

Generic circular arc helper

```

201: bool run(Drivetrain& drivetrain, float errorMagnitude) {
202:     if (hasFailed()) {
203:         drivetrain.set_drive_power(0, 0);
204:         return false;
205:     }
206:
207:     if (!isActive()) {
208:         return false;
209:     }
210:
211:     float now = master_timer.time(msec);
212:     float elapsed = now - stageStartTime;
213:
214:     if (stage == UNSTUCK_ARC_FORWARD_RIGHT) {
215:         applyArc(drivetrain, config.forward_speed, config.turn_speed);
216:
217:         if (elapsed >= config.forward_time) {
218:             stage = UNSTUCK_ARC_FORWARD_LEFT;
219:             stageStartTime = now;
220:             resetMonitor(drivetrain, errorMagnitude);
221:         }
222:
223:         return true;
224:     }
225:
226:     if (stage == UNSTUCK_ARC_FORWARD_LEFT) {
227:         applyArc(drivetrain, config.forward_speed, -config.turn_speed);
228:
229:         if (elapsed >= config.forward_time) {
230:             stage = UNSTUCK_ARC_BACK_RIGHT;
231:             stageStartTime = now;
232:             resetMonitor(drivetrain, errorMagnitude);
233:         }
234:
235:         return true;
236:     }
237:
238:     if (stage == UNSTUCK_ARC_BACK_RIGHT) {
239:         applyArc(drivetrain, -config.reverse_speed, config.turn_speed);
240:
241:         if (elapsed >= config.reverse_time) {
242:             stage = UNSTUCK_ARC_BACK_LEFT;
243:             stageStartTime = now;
244:             resetMonitor(drivetrain, errorMagnitude);
245:         }
246:
247:         return true;
248:     }
249:
250:     if (stage == UNSTUCK_ARC_BACK_LEFT) {
251:         applyArc(drivetrain, -config.reverse_speed, -config.turn_speed);
252:
253:         if (elapsed >= config.reverse_time) {
254:             stage = UNSTUCK_ARC_IDLE;
255:             resetMonitor(drivetrain, errorMagnitude);

```

Function / item	File	How it works
DriveStraightCommand::initialize	include/DriveStraightCommand.h	Sets drive-straight state. Stores initial encoder position, resets PID state, sets timeout, configures UnstuckArcHelper.

Function / item	File	How it works
DriveStraightCommand::execute	include/DriveStraightCommand.h	Encoder-distance PID. Uses wheel encoder distance for linear error and inertial heading for angular correction.
InertialTurnCommand::execute	include/InertialTurnCommand.h	Heading PID turn. Computes shortest heading error, clamps turn output, uses settle timer and unstuck helper.
DriveToX/YPositionCommand::execute	include/DriveToXPositionCommand.h / DriveToYPositionCommand.h	Axis-specific GPS drive. Uses GPS x or y error projected onto the requested heading, while continuously correcting heading.
DriveToPointCommand::execute	include/DriveToPointCommand.h	Point then drive. First turns to face the target, then drives straight toward it. Reverse mode adds 180 degrees to heading and negates linear speed.
DriveToPointUntilX/YCommand::crossedExitLine	include/DriveToPointUntilXCommand.h / DriveToPointUntilYCommand.h	Exit-line variant. Finishes when the robot crosses a specified x or y boundary instead of waiting to settle at a point.
GrabBlockCommand::execute	include/GrabBlockCommand.h	Short encoder drive to collect. Sets target heading to current heading at initialize, drives the requested distance, and uses UnstuckArcHelper if progress stalls.
UnstuckArcHelper::shouldStart	include/UnstuckArcHelper.h	Generic stuck detector. Checks time since last progress and compares encoder, heading, and error progress signals.
UnstuckArcHelper::run	include/UnstuckArcHelper.h	Generic escape sequence. Runs four arc stages with configured speeds/times and returns true while it owns drivetrain output.

8. GPS Filtering and Wall Alignment

The GPS filter rejects samples that jump too far from a reference sample. Wall alignment uses rear distance sensors and GPS heading to drive backward toward a wall while continuously correcting heading.

Filtered GPS sample logic

```

128:         void execute() override{
129:             GetGPSCoordinatesFilteredVar& var = getGPSCoordinatesFilteredVar;
130:             GetGPSCoordinatesFilteredConfig& conf = getGPSCoordinatesFilteredConfig;
131:             float now = master_timer.time();
132:
133:             if (now < var.nextSampleTime){
134:                 return;
135:             }
136:
137:             var.currentX = positionTracking.get_raw_x();
138:             var.currentY = positionTracking.get_raw_y();
139:             var.currentHeading = positionTracking.get_raw_heading();
140:
141:             var.xDifference = var.currentX - var.referenceX;
142:             var.yDifference = var.currentY - var.referenceY;
143:
144:             var.positionalDifference = sqrt(var.xDifference * var.xDifference +
var.yDifference * var.yDifference);
145:
146:             var.headingDifference = headingError(var.currentHeading, var.referenceHeading);
147:
148:             var.positionValid = fabs(var.positionalDifference) <= conf.maxPositionJump;
149:             var.headingValid = fabs(var.headingDifference) <= conf.maxHeadingJump;
150:
151:             if(var.positionValid && var.headingValid){
152:                 var.sumX += var.currentX;
153:                 var.sumY += var.currentY;
154:                 var.sumHeadingDelta += var.headingDifference;
155:                 var.acceptedSamples++;
156:             }
157:
158:             var.samplesTaken++;
159:             var.nextSampleTime = now + conf.sampleIntervals;
160:
161:             if(var.samplesTaken >= conf.sampleCount){
162:                 finished = true;
163:
164:                 if(var.acceptedSamples >= conf.minAcceptedSamples){
165:                     var.filteredX = var.sumX / var.acceptedSamples;
166:                     var.filteredY = var.sumY / var.acceptedSamples;
167:                     var.averageHeadingDelta = var.sumHeadingDelta / var.acceptedSamples;
168:
169:                     var.filteredHeading = normalizeHeading(var.referenceHeading +
var.averageHeadingDelta);
170:
171:
172:             positionTracking.set_pose(var.filteredX, var.filteredY, var.filteredHeading);
173:                 successful = true;
174:             } else {
175:                 successful = false;
176:             }
177:
178:
179:         }
180:
181:         bool isFinished() override{
182:             return finished;
183:         }
184:
185:         void end() override{}
186:
187:

```

Wall alignment sensor averaging and display

```
211:         var.left_sensor_detected = false;
212:         var.right_sensor_detected = false;
213:         var.wall_detected = false;
214:
215:         unstuck.configure(config.unstuck);
216:         unstuck.initialize(drivetrain, std::fabs(target.target_distance));
217:     }
218:
219:     void execute() override {
220:         positionTracking.update_raw_pose();
221:         var.current_heading = positionTracking.get_heading();
222:         var.angular_error =
223:             headingError(target.target_heading, var.current_heading);
224:
225:         var.angular_derivative_error =
226:             var.angular_error - var.angular_previous_error;
227:         var.angular_previous_error = var.angular_error;
228:
229:         var.left_sensor_detected = leftDistanceSensor.isObjectDetected();
230:         var.right_sensor_detected = rightDistanceSensor.isObjectDetected();
231:         var.wall_detected =
232:             var.left_sensor_detected || var.right_sensor_detected;
233:
234:         if (var.left_sensor_detected) {
235:             var.left_distance = leftDistanceSensor.objectDistance(inches);
236:         }
237:
238:         if (var.right_sensor_detected) {
239:             var.right_distance = rightDistanceSensor.objectDistance(inches);
240:         }
241:
242:         if (var.left_sensor_detected && var.right_sensor_detected) {
243:             var.average_distance =
244:                 (var.left_distance + var.right_distance) / 2.0f;
245:         } else if (var.left_sensor_detected) {
246:             var.average_distance = var.left_distance;
247:         } else if (var.right_sensor_detected) {
248:             var.average_distance = var.right_distance;
249:         } else {
250:             var.average_distance = target.target_distance;
251:         }
252:
253:         if (var.wall_detected) {
254:             setLastWallAlignmentDistance(var.average_distance);
255:         }
```

Wall alignment drive output

```

323:         if (!var.wall_detected) {
324:             var.left_drive_linear_velocity = config.max_linear_speed;
325:             var.right_drive_linear_velocity = config.max_linear_speed;
326:         }
327:
328:         bool headingTooFarForDistanceCorrection =
329:             config.max_linear_heading_error > 0.0f &&
330:             std::fabs(var.angular_error) > config.max_linear_heading_error;
331:
332:         if (headingTooFarForDistanceCorrection && var.wall_detected) {
333:             var.left_drive_linear_velocity = 0;
334:             var.right_drive_linear_velocity = 0;
335:         }
336:
337:         // Rear sensors: if the robot is too far from the wall, drive backward.
338:         var.left_drive_linear_velocity *= -1.0f;
339:         var.right_drive_linear_velocity *= -1.0f;
340:
341:         applyAccelLimit();
342:
343:         var.left_drive_linear_velocity =
344:             clamp(var.left_drive_linear_velocity, config.max_linear_speed);
345:         var.right_drive_linear_velocity =
346:             clamp(var.right_drive_linear_velocity, config.max_linear_speed);
347:
348:         float leftPower =
349:             var.left_drive_linear_velocity + var.left_drive_angular_velocity;
350:         float rightPower =
351:             var.right_drive_linear_velocity + var.right_drive_angular_velocity;
352:
353:         scaleDrivePower(leftPower, rightPower);
354:
355:         bool tryingToAlign =
356:             (
357:                 std::fabs(var.linear_error) > config.acceptable_error ||
358:                 std::fabs(var.angular_error) > 3.0f
359:             ) &&
360:             (
361:                 std::fabs(leftPower) > 5.0f ||
362:                 std::fabs(rightPower) > 5.0f
363:             );
364:
365:         if (unstuck.shouldStart(drivetrain, progressError, tryingToAlign)) {
366:             if (!unstuck.start(drivetrain, progressError)) {
367:                 var.timed_out = true;
368:                 finished = true;
369:             }
370:
371:             return;
372:         }
373:
374:         drivetrain.set_drive_power(leftPower, rightPower);
375:
376:         var.current_time = master_timer.time(msec);
377:
378:         bool alignedToWall =
379:             var.wall_detected &&
380:             std::fabs(var.linear_error) < config.acceptable_error &&
381:             std::fabs(var.angular_error) < config.heading_acceptable_error;
382:
383:         if (alignedToWall) {
384:             if (!var.threshold_timer_set) {
385:                 var.inside_threshold_timer =
386:                     var.current_time + config.settle_time;
387:                 var.threshold_timer_set = true;
388:             }
389:
390:             if (var.current_time >= var.inside_threshold_timer) {
391:                 var.at_target = true;
392:             }
393:         } else {
394:             var.threshold_timer_set = false;
395:             var.at_target = false;
396:         }
397:

```

Score approach GPS retry gate

```

634: bool robotIsCloseToPoint(float targetX, float targetY, float acceptableDistance) {
635:     if (!updateGPS.wasSuccessful()) {
636:         return false;
637:     }
638:
639:     float xError = targetX - positionTracking.get_x();
640:     float yError = targetY - positionTracking.get_y();
641:     float distance = std::sqrt(xError * xError + yError * yError);
642:
643:     return distance <= acceptableDistance;
644: }
645:
646: bool scoreBottomLeftApproachIsClose() {
647:     return robotIsCloseToPoint(
648:         scoreBottomLeftTarget.target_x,
649:         scoreBottomLeftTarget.target_y,
650:         scoreApproachRetryDistance
651:     );
652: }
653:
654: bool scoreBottomRightApproachIsClose() {
655:     return robotIsCloseToPoint(
656:         scoreBottomRightTarget.target_x,
657:         scoreBottomRightTarget.target_y,
658:         scoreApproachRetryDistance
659:     );
660: }
661:
662: bool scoreTopLeftApproachIsClose() {
663:     return robotIsCloseToPoint(
664:         scoreTopLeftTarget.target_x,
665:         scoreTopLeftTarget.target_y,
666:         scoreApproachRetryDistance
667:     );
668: }
669:
670: bool scoreTopRightApproachIsClose() {
671:     return robotIsCloseToPoint(
672:         scoreTopRightTarget.target_x,
673:         scoreTopRightTarget.target_y,
674:         scoreApproachRetryDistance
675:     );

```

Function / item	File	How it works
GetGPSCoordinatesFilteredCommand::initialize	include/GetGPSCoordinatesFilteredCommand.h	Starts a GPS sample batch. Resets sums, reference pose, sample counters, and success flag.
GetGPSCoordinatesFilteredCommand::execute	include/GetGPSCoordinatesFilteredCommand.h	Filters GPS data. Collects samples at intervals, accepts samples near the reference, averages accepted x/y and heading delta, and writes filtered pose only if enough samples pass.
wasSuccessful	include/GetGPSCoordinatesFilteredCommand.h	Reports filter result. Used by SetDrivetrainPoseFromGPSCommand and score retry checks to avoid trusting stale GPS data.
SetDrivetrainPoseFromGPSCommand::initialize	src/Routines.cpp	Applies filtered pose. Skips if filtering failed, rejects non-finite pose values, normalizes heading, then updates drivetrain odom pose.
WallAlignmentCommand::execute	include/WallAlignmentCommand.h	Distance and heading alignment. Reads both rear distance sensors, uses their average if both see a wall, computes corrected distance by cosine of heading error, then combines linear and angular PID.
WallAlignmentCommand no-wall behavior	include/WallAlignmentCommand.h	Drives backward if no wall is seen. When neither sensor detects an object, it commands max linear speed before rear-sensor inversion, causing the robot to back toward the wall.
setLastWallAlignmentDistance	src/CommandStatus.cpp	Stores final wall reading. WallAlignmentCommand updates this whenever either rear sensor sees the wall; Brain row 7 keeps the last reading after the command ends.

9. Intake, Color Sorting, Unjamming, and Scoring

Intake owns three intake motors and the optical sensor. The sorting step decides whether the visible object is alliance color or wrong color, counts accepted blocks, rejects wrong-color blocks, and triggers unjamming based on intake velocity drop.

Intake motor modes

```

23: void Intake::intaking(){
24:     Intake::set_intake_power(100,0,100);
25: }
26:
27: void Intake::unjamming(){
28:     Intake::set_intake_power(-100,100,100);
29: }
30:
31: void Intake::color_sorting(){
32:     Intake::set_intake_power(100,0,-100);
33: }
34:
35: void Intake::score_high(){
36:     Intake::set_intake_power(100,-100,100);
37: }
38:
39: void Intake::score_mid(){
40:     Intake::set_intake_power(100,-100,-100);
41: }
42:
43: void Intake::set_blocks_before_scoring(int block_count){

```

Color sorting and block counting

```

169:     if (var.correct_block_detected && !var.accepting_timer_set) {
170:         if (!counting_current_object) {
171:             accepted_block_count++;
172:             counting_current_object = true;
173:         }
174:
175:         var.accepting_end_time = now + conf.accept_time;
176:         var.accepting_timer_set = true;
177:         var.accepting_correct_block = true;
178:     }
179:
180:     if (var.accepting_timer_set) {
181:         if (now < var.accepting_end_time) {
182:             var.accepting_correct_block = true;
183:         } else {
184:             var.accepting_timer_set = false;
185:             var.accepting_correct_block = false;
186:         }
187:     }
188:
189:     if (var.sorting_out_block && !var.sorting_out_block_timer_set) {
190:         var.sorting_end_time = now + conf.sorting_time;
191:         var.done_with_color_sorting = false;
192:         var.sorting_out_block_timer_set = true;
193:
194:         // Wrong color overrides accepting
195:         var.accepting_timer_set = false;
196:         var.accepting_correct_block = false;
197:     }
198:
199:     if (var.sorting_out_block_timer_set) {
200:         if (now < var.sorting_end_time) {
201:             var.done_with_color_sorting = false;
202:         } else {
203:             var.sorting_out_block_timer_set = false;
204:             var.done_with_color_sorting = true;
205:             var.sorting_out_block = false;
206:
207:             // After sorting, force unjamming to re-check from scratch
208:             var.unjamming_timer_set = false;
209:             var.done_with_jamming = true;
210:             var.threshold_velocity_passed = false;
211:         }
212:     } else {
213:         var.done_with_color_sorting = true;

```

Unjam trigger and priority

```

216:     float intakeVelocity = fabs(initial_intake.velocity(pct));
217:
218:     if (var.done_with_color_sorting &&
219:         intakeVelocity > conf.threshold_velocity) {
220:
221:         var.threshold_velocity_passed = true;
222:     }
223:
224:     if ((intakeVelocity < conf.threshold_velocity) &&
225:         (!var.unjamming_timer_set) &&
226:         (var.threshold_velocity_passed) &&
227:         (var.done_with_color_sorting)) {
228:
229:         var.unjamming_end_time = now + conf.unjamming_time;
230:         var.unjamming_timer_set = true;
231:         var.done_with_jamming = false;
232:         var.threshold_velocity_passed = false;
233:     }
234:
235:     if (var.unjamming_timer_set) {
236:         if (now < var.unjamming_end_time) {
237:             var.done_with_jamming = false;
238:             var.threshold_velocity_passed = false;
239:         } else {
240:             var.unjamming_timer_set = false;
241:             var.done_with_jamming = true;
242:             var.threshold_velocity_passed = false;
243:         }
244:     } else {
245:         var.done_with_jamming = true;
246:     }
247:
248:     // -----
249:     // Final behavior priority
250:     // -----
251:
252:     if (!var.done_with_color_sorting) {
253:         // Wrong color was detected, actively reject it
254:         color_sorting();
255:     }
256:     else if (!var.done_with_jamming) {
257:         // Intake appears jammed, regardless of whether a block is currently detected
258:         unjamming();
259:     }
260:     else if (!var.accepting_correct_block) {
261:         // Default state: reject/sort unless correct color has been detected
262:         color_sorting();
263:     }
264:     else {
265:         // Correct block was detected and accept timer is active
266:         intaking();
267:     }
268:
269:     return false;
270: }
271:
272: void Intake::apply_motor_power(){
273:     initial_intake.spin(fwd, initial_intake_power, pct);

```

Function / item	File	How it works
set_intake_power	src/Intake.cpp	Stores motor percentages. Like drivetrain, motor outputs are stored first and applied in main().
intaking / unjamming / color_sorting	src/Intake.cpp	Preset motor modes. Set motor power patterns for accepting, reversing/unjamming, or rejecting wrong color.
score_high / score_mid	src/Intake.cpp	Scoring modes. Run motor combinations used by WaitAndScoreCommand.
set_blocks_before_scoring	src/Intake.cpp	Sets scoring threshold. Clamps target block count to at least 1 unless the command only wants to reset count.
intake_with_sorting_init	src/Intake.cpp	Resets sorting state. Loads config, sets default motor powers, resets object/color/timer/jam variables.

Function / item	File	How it works
intake_with_sorting_step	src/Intake.cpp	Main intake logic. Reads hue, classifies red/blue, counts accepted blocks once per object, times wrong-color rejection, detects jams from velocity drop, and chooses final motor mode by priority.
IntakeWithSorting::execute	include/IntakeWithSortingCommand.h	Command wrapper. Calls intake.intake_with_sorting_step() each scheduler loop.
WaitAndScoreCommand::execute	include/WaitAndScoreCommand.h	Timed scoring command. Waits for configured delay, runs score_high or score_mid for score_time, then stops the intake.
SetBlocksBeforeScoringCommand::initialize	include/SetBlocksBeforeScoringCommand.h	Sets/reset count. Optionally sets blocks_before_scoring and resets accepted count when reset_count is true.

10. Autonomous Routine Assembly

Routines.cpp is the hub where configs, targets, command objects, conditional checks, and command sequences are assembled. AI_ROUTE_ONE repeats AI_ROUTE_1 forever, so after one collect/score cycle ends it starts again.

Main repeated autonomous route

```

1135:     AI_ROUTE_1.addCommand(&set_blocks_before_scoring);
1136:
1137:     AI_ROUTE_1.addCommand(&updateGPS);
1138:     AI_ROUTE_1.addCommand(&setPoseFromGPS);
1139:
1140:     AI_ROUTE_1.addCommand(&find_block);
1141:
1142:     AI_ROUTE_1.addCommand(&updateGPS);
1143:     AI_ROUTE_1.addCommand(&setPoseFromGPS);
1144:
1145:     AI_ROUTE_1.addCommand(&track_and_intake);
1146:
1147:     AI_ROUTE_1.addCommand(&choose_grab_or_skip);
1148:
1149:     AI_ROUTE_1.addCommand(&choose_score_or_keep_collecting);
1150: }
```

Scoring sequence retry before wall alignment

```

1083:     score_bottom_left_sequence.addCommand(&drive_to_score_bottom_left);
1084:     score_bottom_left_sequence.addCommand(&updateGPS);
1085:     score_bottom_left_sequence.addCommand(&setPoseFromGPS);
1086:     score_bottom_left_sequence.addCommand(&retry_bottom_left_score_drive_if_needed);
1087:     score_bottom_left_sequence.addCommand(&turn_for_bottom_left_wall_alignment);
1088:     score_bottom_left_sequence.addCommand(&align_to_bottom_left_wall);
1089:     score_bottom_left_sequence.addCommand(&turn_for_left_scoring);
1090:     score_bottom_left_sequence.addCommand(&reverse_to_score_bottom_left);
1091:     score_bottom_left_sequence.addCommand(&drive_into_left_goal);
1092:     score_bottom_left_sequence.addCommand(&wait_and_score);
1093:     score_bottom_left_sequence.addCommand(&reset_blocks_after_scoring);
1094:
1095:     score_bottom_right_sequence.addCommand(&drive_to_score_bottom_right);
1096:     score_bottom_right_sequence.addCommand(&updateGPS);
1097:     score_bottom_right_sequence.addCommand(&setPoseFromGPS);
1098:     score_bottom_right_sequence.addCommand(&retry_bottom_right_score_drive_if_needed);
1099:     score_bottom_right_sequence.addCommand(&turn_for_bottom_right_wall_alignment);
1100:     score_bottom_right_sequence.addCommand(&align_to_bottom_right_wall);
1101:     score_bottom_right_sequence.addCommand(&turn_for_right_scoring);
1102:     score_bottom_right_sequence.addCommand(&reverse_to_score_bottom_right);
1103:     score_bottom_right_sequence.addCommand(&drive_into_right_goal);
1104:     score_bottom_right_sequence.addCommand(&wait_and_score);
1105:     score_bottom_right_sequence.addCommand(&reset_blocks_after_scoring);
1106:
1107:     score_top_left_sequence.addCommand(&drive_to_score_top_left);
1108:     score_top_left_sequence.addCommand(&updateGPS);
1109:     score_top_left_sequence.addCommand(&setPoseFromGPS);
1110:     score_top_left_sequence.addCommand(&retry_top_left_score_drive_if_needed);
1111:     score_top_left_sequence.addCommand(&turn_for_top_left_wall_alignment);
1112:     score_top_left_sequence.addCommand(&align_to_top_left_wall);
1113:     score_top_left_sequence.addCommand(&turn_for_left_scoring);
1114:     score_top_left_sequence.addCommand(&reverse_to_score_top_left);
1115:     score_top_left_sequence.addCommand(&drive_into_left_goal);
1116:     score_top_left_sequence.addCommand(&wait_and_score);
1117:     score_top_left_sequence.addCommand(&reset_blocks_after_scoring);
1118:
1119:     score_top_right_sequence.addCommand(&drive_to_score_top_right);
1120:     score_top_right_sequence.addCommand(&updateGPS);
1121:     score_top_right_sequence.addCommand(&setPoseFromGPS);
1122:     score_top_right_sequence.addCommand(&retry_top_right_score_drive_if_needed);
1123:     score_top_right_sequence.addCommand(&turn_for_top_right_wall_alignment);
1124:     score_top_right_sequence.addCommand(&align_to_top_right_wall);
1125:     score_top_right_sequence.addCommand(&turn_for_right_scoring);
1126:     score_top_right_sequence.addCommand(&reverse_to_score_top_right);
1127:     score_top_right_sequence.addCommand(&drive_into_right_goal);
1128:     score_top_right_sequence.addCommand(&wait_and_score);
1129:     score_top_right_sequence.addCommand(&reset_blocks_after_scoring);
1130:
1131:     score_sequence.addCommand(&updateGPS);
1132:     score_sequence.addCommand(&setPoseFromGPS);

```

Function / item	File	How it works
trackBlockWasSuccessful	src/Routines.cpp	Branch helper. Returns true only if the last TrackBlockCommand ended with TRACK_BLOCK_SUCCESS.
hasEnoughBlocksToScore	src/Routines.cpp	Branch helper. Returns intake.has_enough_blocks_to_score().
robotIsOnTopSide / robotIsOnRightSide	src/Routines.cpp	Corner selector helpers. Use the current filtered/cached pose to choose which scoring corner sequence to run.
robotIsCloseToPoint	src/Routines.cpp	Score approach retry check. Requires the latest filtered GPS command to have succeeded, then checks Euclidean distance to the approach point.
score*ApproachIsClose	src/Routines.cpp	Per-corner retry conditions. Call robotIsCloseToPoint() with the relevant reflected score approach target.
build_AI_routine	src/Routines.cpp	Builds command graph. Adds intake helpers to parallel groups, builds all four scoring sequences, then builds the repeated AI route.

Function / item	File	How it works
choose_score_corner	src/Routines.cpp	Nested conditional. First chooses top vs bottom by y, then left vs right by x.

11. Full Function Reference by File

This section is the compact function-by-function index. Earlier sections show the most important snippets; this index names what each function or item is responsible for.

Core command framework

Function / item	File	How it works
Command::initialize	include/Command.h	Optional setup hook. Default empty implementation used before first execute().
Command::execute	include/Command.h	Required loop hook. Pure virtual; each command defines one scheduler-cycle slice.
Command::isFinished	include/Command.h	Required completion hook. Pure virtual; scheduler removes the command once true.
Command::end	include/Command.h	Optional cleanup hook. Default empty; most motor commands stop outputs here.
Scheduler::removeCommand	include/Scheduler.h	Internal erase helper. Removes command and matching initialized flag.
Scheduler::schedule	include/Scheduler.h	Queue command. Adds non-null command pointer with initialized=false.
Scheduler::run	include/Scheduler.h	Command loop. Initializes, executes, ends, and removes finished commands.
Scheduler::cancelAll	include/Scheduler.h	Stop all scheduled work. Ends initialized commands then clears vectors.
Scheduler::isEmpty	include/Scheduler.h	Status helper. Returns whether any commands are scheduled.

Command groups

Function / item	File	How it works
<code>SequentialCommandGroup::addCommand</code>	<code>src/SequentialCommandGroup.cpp</code>	Append child. Adds non-null command to list.
<code>SequentialCommandGroup::initialize</code>	<code>src/SequentialCommandGroup.cpp</code>	Start sequence. Resets index and finished state.
<code>SequentialCommandGroup::execute</code>	<code>src/SequentialCommandGroup.cpp</code>	Run current child. Initializes current child, executes it, ends it when finished, then advances.
<code>SequentialCommandGroup::end</code>	<code>src/SequentialCommandGroup.cpp</code>	End active child. If interrupted, calls <code>end()</code> on current child.
<code>ParallelCommandGroup::execute</code>	<code>src/ParallelCommandGroup.cpp</code>	Run all children. Executes unfinished children until all are complete.
<code>ParallelDeadlineGroup::execute</code>	<code>src/ParallelDeadlineGroup.cpp</code>	Run deadline and helpers. Group finishes when deadline finishes.
<code>RaceCommandGroup::execute</code>	<code>src/RaceCommandGroup.cpp</code>	Race children. Group finishes as soon as any child finishes.
<code>ConditionalCommandGroup::initialize</code>	<code>src/ConditionalCommandGroup.cpp</code>	Select branch. Chooses <code>trueCommand</code> or <code>falseCommand</code> and initializes it.
<code>RepeatForeverCommandGroup::execute</code>	<code>src/RepeatForeverCommandGroup.cpp</code>	Restart wrapped command. When child finishes, ends and reinitializes it.

Drivetrain and pose

Function / item	File	How it works
main	src/main.cpp	Program entry point. Sets up Jetson, builds/schedules autonomous, waits for calibration, seeds odom from GPS, then loops scheduler/motors/display.
Drivetrain::get_pose	src/Drivetrain.cpp	Return cached pose. Returns the internal Pose struct.
Drivetrain::set_odom_pose	src/Drivetrain.cpp	Set odom pose. Stores x/y/theta and updates inertial heading.
Drivetrain::set_heading_degrees	src/Drivetrain.cpp	Set inertial heading safely. Rejects invalid values, normalizes, then writes to sensor.
Drivetrain::update_odom_pose	src/Drivetrain.cpp	Refresh heading. Copies inertial heading into pose.theta.
Drivetrain::set_drive_power	src/Drivetrain.cpp	Store drive command. Updates left_drive_power/right_drive_power.
Drivetrain::apply_motor_power	src/Drivetrain.cpp	Spin motors. Applies stored power to motor groups.
Drivetrain::stop	src/Drivetrain.cpp	Brake drivetrain. Zeros power and calls stop(brake).
PositionTracking::normalize_heading	src/PositionTracking.cpp	Normalize heading. Wraps heading to 0..360.
PositionTracking::get_raw_x/y	src/PositionTracking.cpp	Corrected GPS x/y. Converts sensor position to robot-center position using heading and sensor offset.
PositionTracking::get_raw_heading	src/PositionTracking.cpp	Corrected heading convention. Returns GPS heading plus 180 normalized.
PositionTracking::update_raw_pose	src/PositionTracking.cpp	Cache GPS pose. Stores corrected raw x/y/heading.
PositionTracking::set_pose	src/PositionTracking.cpp	Set filtered pose. Stores filtered x/y/heading into gpsPose.
PositionTracking getters	src/PositionTracking.cpp	Access pose fields. Return cached x, y, or heading.

Vision and block tracking

Function / item	File	How it works
JetsonSerial::JetsonSerialSetup	src/JetsonSerial.cpp	Open serial input. Opens /dev/serial1, reports status on Brain screen, seeds rand.
JetsonSerial::clear_blocks	src/JetsonSerial.cpp	Reset block arrays. Clears block count, selected block, and arrays.
JetsonSerial::parse_xy_pair	src/JetsonSerial.cpp	Parse x,y token. Splits a string pair and converts to integers.
JetsonSerial::update_block_pose	src/JetsonSerial.cpp	Read Jetson frame. Parses 0, x,y, or count;x,y,... messages.
FindBlockCommand helper functions	include/FindBlockCommand.h	Allowed find/center logic. Estimate field position, filter avoid zones, select allowed candidates, monitor search/centering progress, and run state machine.
TrackBlockCommand helper functions	include/TrackBlockCommand.h	Allowed tracking logic. Track only acceptable blocks, drive with PID, and use circular arc unstuck sequence when progress stalls.

Movement and scoring commands

Function / item	File	How it works
DriveStraightCommand::execute	include/DriveStraightCommand.h	Distance and heading PID. Uses encoder distance and inertial heading to drive a straight target.
InertialTurnCommand::execute	include/InertialTurnCommand.h	Turn PID. Turns to target heading with settle time and unstuck support.
DriveToX/YPositionCommand::execute	include/DriveToXPositionCommand.h / DriveToYPositionCommand.h	Axis target drive. Uses GPS axis error and heading correction.
DriveToPointCommand helpers	include/DriveToPointCommand.h	Point-drive state machine. Compute target heading/distance, point first, drive second, and optionally escape with arcs.
DriveToPointUntilX/YCommand helpers	include/DriveToPointUntilXCommand.h / include/DriveToPointUntilYCommand.h	Exit-line state machine. Same as drive-to-point, but finish when crossing an x or y line.
GrabBlockCommand::execute	include/GrabBlockCommand.h	Final block grab motion. Short controlled drive after tracking aligns to a block.
WallAlignmentCommand::execute	include/WallAlignmentCommand.h	Wall distance alignment. Uses rear distance sensors and GPS heading to back to target wall distance while correcting heading.
UnstuckArcHelper functions	include/UnstuckArcHelper.h	Reusable escape support. Configure, monitor progress, start, run, and fail a four-stage circular arc escape.

Intake and scoring

Function / item	File	How it works
Intake::intake_with_sorting_step	src/Intake.cpp	Color sorting state machine. Reads optical sensor, classifies color, counts accepted blocks, rejects wrong color, and unjams on velocity drop.
Intake::apply_motor_power	src/Intake.cpp	Spin intake motors. Applies stored powers to initial/middle/final intake motors.
IntakeWithSorting::execute	include/IntakeWithSortingCommand.h	Sorting command wrapper. Runs one intake sorting step.
WaitAndScoreCommand::execute	include/WaitAndScoreCommand.h	Timed scoring. Waits, scores, then stops.
SetBlocksBeforeScoringCommand::initialize	include/SetBlocksBeforeScoringCommand.h	Configure score threshold. Sets target block count and optionally resets accepted count.

Routines and debug

Function / item	File	How it works
trackBlockWasSuccessful	src/Routines.cpp	Track result condition. Used to decide whether to run GrabBlockCommand.
hasEnoughBlocksToScore	src/Routines.cpp	Score condition. Used to decide whether to run scoring sequence.
robotIsOnTopSide / robotIsOnRightSide	src/Routines.cpp	Field side conditions. Choose the correct reflected scoring corner.
robotIsCloseToPoint and score*ApproachIsClose	src/Routines.cpp	Pre-wall-alignment retry checks. Use latest filtered GPS pose to decide whether to retry approach drive.
build_AI_routine	src/Routines.cpp	Command graph builder. Creates parallel intake groups, scoring sequences, and repeated autonomous sequence.
setCommandStatus / getCommandStatus	src/CommandStatus.cpp	Debug text storage. Stores the command label printed on Brain row 12.
setLastWallAlignmentDistance / getLastWallAlignmentDistance	src/CommandStatus.cpp	Wall distance display. Stores the last distance sensor reading printed on Brain row 7.

12. Explanation Checklist for Presenting the Code

Use this checklist when explaining the code to someone else.

- Start with the scheduler: the robot is not a single blocking while-loop; each command advances one slice per scheduler cycle.
- Explain the autonomous loop: set block target, GPS update, find block, GPS update, track/intake, grab if track succeeded, score if enough blocks.
- Explain pose: GPS readings are corrected from the physical sensor location to robot-center coordinates before other commands use them.
- Explain block filtering: visible blocks are converted to estimated field positions; avoid-zone blocks are skipped before selecting a target.
- Explain find behavior: normal find rotates in place; circular arc unstuck only starts if heading does not change while trying to rotate.
- Explain tracking behavior: x error controls turn, y error controls forward motion, and progress is measured by encoder, heading, and y-error changes.
- Explain scoring: after driving toward the scoring approach point, the robot re-runs filtered GPS and retries the drive once if it is not close enough.
- Explain wall alignment: rear sensors report wall distance, heading is corrected continuously, and the robot backs toward the wall if neither sensor sees it.
- Explain intake: color sorting rejects wrong color, counts accepted blocks, and can unjam based on velocity drop even if it is not currently detecting a block.
- Explain debug screen: selected block, errors, wall distance, GPS/IMU headings, and current command status are printed every loop.